



**UNIVERSIDAD DE CONCEPCIÓN
FACULTAD DE INGENIERÍA
DEPARTAMENTO DE INGENIERÍA INFORMÁTICA Y CIENCIAS DE LA
COMPUTACIÓN**

Informe Técnico

Proyecto Final – Redes de Computadores

Sistema de monitoreo remoto para un entorno ciberfísico simulado

8 de julio de 2026
Concepción (Chile)

Antonio Jesus Benavides Puentes

Actuadores y Grafana

2023455954

antbenavides2023@udec.cl

Valeria Paulette Quiroga Carrere

Broker

2023453609

vquiroga2023@udec.cl

Bastián Alexander Bernal Villegas

Publicadores

2023418871

bbernal2022@udec.cl

Andrés Ignacio Chaparro Maldonado

Server

2023452793

achaparro2023@udec.cl

Resumen

Este proyecto presenta el diseño e implementación de un sistema de monitoreo remoto para una planta fotovoltaica ciberfísica simulada, orientado a la supervisión individual de paneles solares con el fin de detectar pérdidas de rendimiento, fallas y condiciones anómalas que no son evidentes mediante indicadores agregados. La solución se desarrolla completamente en software, simulando tanto las condiciones ambientales como el comportamiento eléctrico y el nivel de ensuciamiento de cada panel. La arquitectura propuesta integra nodos publicadores, un broker MQTT, un servidor central para el procesamiento y almacenamiento de datos, actuadores y un sistema de visualización, comunicados a través de una red TCP/IP real mediante el protocolo MQTT. Además, incorpora mecanismos básicos de seguridad, control de acceso y validación de la infraestructura de comunicación. Finalmente, el sistema es evaluado mediante pruebas funcionales y análisis del tráfico de red con Wireshark, verificando la confiabilidad de la comunicación y el correcto funcionamiento de la arquitectura distribuida propuesta.

Índice

1	Problema y objetivos	1
1.1	Problema	1
1.2	Objetivo General	1
1.3	Objetivos Específicos	1
2	Arquitectura del sistema	3
3	Tecnologías utilizadas	4
4	Diseño del flujo de comunicación o protocolo	5
4.1	Justificación del uso de MQTT	5
4.2	Diseño del esquema de tópicos	5
4.3	Formato de los mensajes	6
4.4	Cadencia de publicación de mensajes	6
5	Implementación	8
5.1	Modelo ambiental	8
5.2	Modelo del panel y generación de variables de telemetría	8
5.3	Publicación MQTT y ejecución de la simulación	9
5.4	Actuador	10
5.5	Capa MQTT compartida	10
5.6	Componente de servidor y persistencia	11
6	Seguridad y confiabilidad incorporada	12
6.1	Autenticación cliente–broker	12
6.2	Control de acceso mediante ACL MQTT	12
6.3	Endurecimiento de la infraestructura de visualización	13

7 Pruebas y validación	14
8 Análisis con Wireshark	17
8.1 Análisis de tráfico mediante Wireshark	17
9 Limitaciones del sistema	19
10 Conclusiones	20
A Anexos	21

Índice de figuras

2	Ejecución de publicaciones de los 350 paneles solares.	15
1	Datos de paneles guardados en la base de datos	15
3	Captura de intercambio TCP para establecer la conexión entre un suscriptor (172.31.181.0) y el broker (192.168.50.186).	17
4	Captura de paquete perdido y solicitud para la retransmisión.	17
5	Nmap al Servidor	21
6	Ping al Servidor	21
7	Nmap al Broker	22
8	Tráfico de red en la máquina del servidor	22

1 Problema y objetivos

1.1 Problema

El monitoreo de plantas fotovoltaicas de gran escala presenta un desafío importante: la disminución del rendimiento de un panel fotovoltaico individual puede pasar desapercibida cuando únicamente se dispone de indicadores agregados correspondientes a un conjunto de paneles o a la planta en su totalidad. Esta limitación dificulta la detección temprana de fallas, pérdidas de eficiencia o condiciones anómalas que afectan el desempeño del sistema.

Para abordar este problema es necesario contar con un sistema capaz de recopilar información de cada panel de forma individual, transmitirla de manera confiable hacia un punto central de procesamiento y analizarla para detectar eventos relevantes. Asimismo, resulta deseable que el sistema sea capaz de generar alertas o ejecutar acciones correctivas de manera automática o semiautomática cuando se identifiquen condiciones fuera de los parámetros normales de operación.

Considerando que el proyecto no contempla el uso de hardware físico adicional, la solución se desarrolla íntegramente en software. Para ello, se simula tanto el entorno físico de una planta fotovoltaica (incluyendo variables como radiación solar, temperatura ambiente y nubosidad) como el comportamiento eléctrico y el nivel de ensuciamiento de cada panel. La comunicación entre los diferentes componentes del sistema, conformados por el simulador, el broker MQTT, el servidor central, el actuador y el sistema de visualización, se realiza sobre una red TCP/IP real, permitiendo validar el funcionamiento de una arquitectura distribuida de monitoreo remoto.

1.2 Objetivo General

Diseñar, implementar y validar un sistema de monitoreo remoto para una planta fotovoltaica ciberfísica simulada, basado en una arquitectura distribuida que integre comunicación cliente-servidor mediante el protocolo MQTT, mecanismos básicos de seguridad, almacenamiento de datos y herramientas de visualización y análisis para el monitoreo del sistema.

1.3 Objetivos Específicos

- Diseñar una arquitectura distribuida compuesta por nodos simulados publicadores, un broker MQTT central, un servidor encargado del procesamiento de la información, nodos actuadores y un sistema de visualización de datos.
- Implementar la comunicación entre los distintos componentes utilizando el protocolo MQTT sobre TCP/IP, mediante un cliente asíncrono basado en `aiomqtt` y un broker `Mosquitto`.

- Incorporar mecanismos básicos de seguridad, incluyendo autenticación de clientes y control de acceso a los distintos tópicos de comunicación.
- Implementar un mecanismo de confiabilidad mediante un nodo actuador capaz de ejecutar acciones correctivas en respuesta a alertas o comandos recibidos a través de la red.
- Almacenar la información generada por el sistema en una base de datos PostgreSQL.
- Implementar un sistema de visualización histórica y en tiempo real mediante Grafana para facilitar el monitoreo del estado de la planta simulada.
- Validar el funcionamiento de la arquitectura de red y la exposición de los servicios mediante herramientas de análisis y diagnóstico como Wireshark y Nmap.

2 Arquitectura del sistema

El sistema sigue una arquitectura de publicación/suscripción centrada en un broker MQTT, con cinco roles lógicos:

- **Nodo simulado:** Un único proceso Python asíncrono modela de forma vectorizada 350 paneles solares en 3 clústeres, cada uno con un desfase horario distinto para simular distintas zonas geográficas. Se optó por esta alternativa en lugar de lanzar muchos procesos independientes con el fin de priorizar el rendimiento y la capacidad de cómputo por sobre el realismo de un proceso por panel. La identidad de cada panel se preserva a nivel de tópico MQTT y de payload.
- **Broker MQTT:** Eclipse Mosquitto, alojado en una máquina virtual Ubuntu Server aprovisionada mediante *autoinstall* (*cloud-init*), resoluble en la red local como **broker-vm** vía **avahi-daemon** (mDNS). Es el único punto de acoplamiento entre publicadores y suscriptores.
- **Servidor:** Suscriptor que escucha todos los tópicos de datos provenientes del panel. Esta encargado de mandar retroalimentación a los actuadores en caso de ser necesario y registrar las mediciones en la base de datos para que el rol visualizador pueda acceder a los datos.
- **Actuador:** Suscriptor especializado que escucha canales de alerta y control para luego traducir los mensajes recibidos en acciones reales como la limpieza sobre cierto panel o incluso de un clúster completo.
- **Visualizador:** Instancia de Grafana en una *Máquina Virtual* separada, destinada a mostrar tableros con el historial de mediciones gracias a la capa de persistencia.

3 Tecnologías utilizadas

El desarrollo del proyecto se realizó utilizando el lenguaje de programación **Python**, debido a su versatilidad, amplia disponibilidad de bibliotecas y facilidad para implementar aplicaciones orientadas a la comunicación en red, el procesamiento de datos y la programación asíncrona.

Para el desarrollo de la aplicación se emplearon las siguientes tecnologías y herramientas:

- **Python:** Lenguaje principal utilizado para implementar la lógica del sistema, el procesamiento de datos y la comunicación entre los distintos componentes.
- **asyncio:** Biblioteca estándar de Python utilizada para implementar programación asíncrona, permitiendo ejecutar múltiples tareas concurrentes de forma eficiente sin bloquear la ejecución del programa.
- **aiomqtt (>= 2.0.0):** Cliente MQTT asíncrono basado en la sintaxis `async/await`, empleado para gestionar la comunicación mediante el protocolo MQTT de manera eficiente y escalable.
- **Mosquitto:** Broker MQTT de código abierto encargado de administrar la comunicación entre los clientes publicadores y suscriptores del sistema.
- **mosquitto-clients:** Conjunto de herramientas de línea de comandos que permite publicar y suscribirse a tópicos MQTT, facilitando las tareas de prueba, depuración y monitoreo de la comunicación.
- **PostgreSQL:** Sistema de gestión de bases de datos relacional utilizado para almacenar de forma persistente la información generada por el sistema. Se seleccionó por su robustez, alto rendimiento, confiabilidad y soporte para consultas complejas.
- **Grafana:** Plataforma de visualización utilizada para crear paneles de monitoreo en tiempo real, permitiendo representar gráficamente las métricas y datos recopilados por el sistema.
- **Avahi Daemon:** Servicio de descubrimiento automático de dispositivos y servicios en redes locales mediante el protocolo mDNS, utilizado para simplificar la detección de servicios dentro de la infraestructura.
- **Ubuntu Server LTS:** Sistema operativo utilizado como plataforma de despliegue del servidor. Se eligió la versión de soporte a largo plazo (LTS) por ofrecer mayor estabilidad, seguridad y mantenimiento continuo para entornos de producción.

4 Diseño del flujo de comunicación o protocolo

4.1 Justificación del uso de MQTT

Se seleccionó MQTT como protocolo de comunicación en lugar de una implementación basada en sockets TCP debido a que el sistema presenta un modelo natural de comunicación basado en publicación/suscripción. En este esquema, múltiples nodos de adquisición (paneles fotovoltaicos simulados) generan datos de telemetría que pueden ser consumidos por diferentes componentes del sistema, tales como el servidor de procesamiento, módulos de persistencia, actuadores o sistemas de visualización.

El uso de MQTT permite desacoplar los productores de información de los consumidores mediante la incorporación de un broker central. De esta forma, los nodos emisores no requieren conocer la cantidad ni la identidad de los clientes conectados, facilitando la escalabilidad del sistema cuando aumenta el número de paneles o aplicaciones consumidoras. Además, que permite conectarse a dispositivos pequeños como microcontroladores, ya que el overhead implementado es pequeño (2 bytes) y funciona bien en sistemas de alta latencia y baja capacidad de red.

Además, MQTT incorpora niveles de calidad de servicio (QoS) configurables según la importancia de cada mensaje y permite el uso de suscripciones mediante comodines (+ y #). Esto facilita, por ejemplo, que un consumidor pueda recibir información de todos los paneles mediante un único patrón de suscripción o acceder únicamente a una categoría específica de datos, como alertas o comandos de control.

4.2 Diseño del esquema de tópicos

La estructura de tópicos fue definida utilizando una jerarquía basada en el tipo de información transmitida y el identificador único del panel:

- `solar_panel_data/{panel_id}`: Contiene un mensaje JSON con todas las variables medidas por un panel específico.
- `solar_panel_data/{panel_id}/temperature`: Publica únicamente la temperatura del panel.
- `solar_panel_data/{panel_id}/power`: Publica la potencia eléctrica generada.
- `solar_panel_data/{panel_id}/irradiance`: Publica la irradiancia recibida por el panel.
- `actuador/#`: Canal utilizado para enviar comandos de control hacia los actuadores.

La publicación del mensaje completo permite reducir la cantidad de transmisiones cuando un consumidor requiere múltiples variables simultáneamente. Por otro lado, los tópicos individuales por métrica permiten realizar suscripciones específicas y reducir el procesamiento cuando solo se requiere una variable determinada.

4.3 Formato de los mensajes

El mensaje principal asociado a un panel contiene todas las variables de telemetría junto con su identificador y marca temporal. El tiempo se representa mediante una marca Unix en segundos con precisión decimal, permitiendo sincronizar los datos independientemente del tiempo real de ejecución de la simulación.

Listing 1: Mensaje combinado publicado en `solar_panel_data/{panel_id}`.

```
1 {  
2   "id_panel": 12,  
3   "tiempo": 1751840000.123,  
4   "potencia": 215.7,  
5   "irradiancia": 731.2,  
6   "temperatura": 46.8  
7 }
```

Para aplicaciones que requieren únicamente una variable específica, se utiliza el formato reducido:

Listing 2: Mensaje individual de una métrica publicado en `solar_panel_data/{panel_id}/power`.

```
1 {  
2   "value": 215.7,  
3   "timestamp": 1751840000.123  
4 }
```

4.4 Cadencia de publicación de mensajes

La simulación utiliza tres parámetros independientes para separar la evolución interna del modelo, la escala temporal simulada y la comunicación mediante red: `tick_rate`, `publish_rate` y `time_scale` definidos en el archivo `config.yaml`.

El parámetro `tick_rate` determina la frecuencia con la que se actualiza el estado interno del modelo de simulación. Por otro lado, `time_scale` establece la relación entre el tiempo simulado y el tiempo real, permitiendo acelerar la ejecución y representar ciclos completos de operación del panel fotovoltaico en un intervalo reducido.

Finalmente, `publish_rate` define la frecuencia de envío de mensajes MQTT hacia el broker. Esta separación evita transmitir cada actualización interna del modelo, reduciendo la

carga de comunicación y permitiendo controlar el volumen de datos almacenados o procesados por los consumidores.

De esta manera, el sistema mantiene una separación clara entre la dinámica de simulación, la generación de datos y la comunicación externa, favoreciendo la escalabilidad y eficiencia del flujo de telemetría.

5 Implementación

La implementación del sistema se estructura en cinco componentes principales: el modelo de simulación de paneles fotovoltaicos (`src/nodes`), la capa de comunicación MQTT (`src/mqtt`), la configuración general del sistema (`src/config`), el servicio de persistencia y procesamiento (`src/server`) y la capa de visualización basada en Grafana (`virtual_machine_configs/grafana`).

5.1 Modelo ambiental

El módulo `src/nodes/environment.py` implementa la clase `EnvModel`, encargada de generar las condiciones ambientales que afectan a los paneles fotovoltaicos simulados. El modelo considera tres fenómenos principales:

- **Ciclo día/noche:** se modela mediante una función sinusoidal con un desfase asociado al clúster al que pertenece cada panel:

$$\text{solar}_i(t) = \max \left(0, \sin \left(\frac{2\pi(t \bmod T)}{T} - \frac{\pi}{2} + \varphi_i \right) \right)$$

donde T representa la duración del día simulado y φ_i corresponde al desfase horario del clúster del panel i . Esto permite representar diferencias de iluminación entre distintas zonas geográficas o grupos de paneles.

- **Temperatura ambiente:** se genera mediante una señal sinusoidal sincronizada con el ciclo solar, permitiendo representar la variación térmica diaria asociada a la radiación recibida.
- **Eventos de sombreado:** se generan mediante un proceso de Poisson, donde la probabilidad de ocurrencia de un evento durante un intervalo dt está dada por:

$$P(\text{evento en } dt) = 1 - e^{-\lambda dt}$$

Cada evento posee una intensidad de bloqueo obtenida mediante una distribución Beta(α, β) y una duración determinada mediante una distribución exponencial con un piso mínimo. Los eventos pueden afectar a un clúster completo o a un subconjunto aleatorio de paneles. En caso de existir múltiples eventos simultáneos, sus efectos se combinan multiplicativamente sobre el factor de sombreado individual de cada panel.

5.2 Modelo del panel y generación de variables de telemetría

El módulo `src/nodes/panels.py`, mediante la clase `PanelSimulator`, representa el comportamiento individual de los paneles fotovoltaicos. El único estado persistente mantenido

por este componente corresponde al factor de suciedad de cada panel (`dirt_factor`), el cual evoluciona mediante una caminata aleatoria con arrastre positivo y se limita al intervalo $[0, 1]$.

A partir del estado de suciedad y de las condiciones ambientales generadas por `EnvModel`, se calculan de forma vectorizada las variables de telemetría asociadas a cada panel:

Listing 3: Cálculo vectorizado de variables de telemetría en `panels.py`.

```
1 irradiance = max_irradiance * solar * shadow \  
2             * (1 - dirt_factor * max_dirt_loss) \  
3 \  
4 irradiance = max(0, irradiance + gaussian_noise) \  
5 \  
6 power = max(0, irradiance * area * efficiency \  
7            + gaussian_noise) \  
8 \  
9 temperature = ambient_temp \  
10               + temp_irradiance_coeff * irradiance \  
11               + gaussian_noise
```

El uso de operaciones vectorizadas permite actualizar simultáneamente todos los paneles sin recurrir a iteraciones explícitas en Python, mejorando el rendimiento de la simulación.

El módulo también proporciona las operaciones `clean_panel` y `clean_panels`, utilizadas por el actuador para restablecer el nivel de suciedad de un panel individual o de un conjunto perteneciente a un mismo clúster.

5.3 Publicación MQTT y ejecución de la simulación

El módulo `src/nodes/publisher.py` implementa la clase `MQTTPublisher`, encargada de gestionar la comunicación con el broker MQTT. Mantiene una conexión persistente mediante un administrador de contexto asíncrono y publica periódicamente los mensajes generados por los 350 paneles simulados.

La publicación de mensajes se realiza de forma concurrente utilizando `asyncio.gather`, evitando que la transmisión de un panel bloquee el envío del resto.

El módulo `src/nodes/simulation.py` contiene la clase `Simulation`, que actúa como coordinador principal del sistema. En cada ciclo de ejecución realiza las siguientes tareas:

1. Actualización del modelo ambiental.
2. Evolución del factor de suciedad de los paneles.
3. Cálculo de las variables de telemetría.
4. Publicación MQTT cuando se alcanza el intervalo definido por `publish_rate`.

Además, expone las operaciones de actuación `clean_panel`, `clean_cluster` y `clear_shadow_events`, permitiendo modificar el estado de la simulación en tiempo de ejecución.

5.4 Actuador

El módulo `src/nodes/actuador.py` implementa la clase `ActuadorClient`, responsable de recibir órdenes de control mediante MQTT. Este componente se suscribe al tópico:

- `actuador/`

Al recibir un mensaje, el actuador interpreta la acción solicitada, la cual puede estar definida explícitamente mediante un cuerpo JSON o inferirse a partir de la estructura jerárquica del tópico MQTT.

Las acciones disponibles corresponden a:

- limpieza de un panel individual (`clean_panel`);
- limpieza de un clúster completo (`clean_cluster`);
- eliminación de eventos de sombreado activos (`clear_shadow_events`).

El componente se ejecuta junto con la simulación dentro del mismo proceso y ciclo de eventos asíncrono, permitiendo modificar el estado del modelo mientras este continúa generando datos.

5.5 Capa MQTT compartida

El directorio `src/mqtt` contiene los elementos comunes utilizados por los distintos componentes de comunicación.

El módulo `broker.py` implementa la clase `Broker`, encargada de centralizar la configuración de conexión al servidor MQTT, incluyendo dirección y puerto definidos mediante `settings.py`.

El módulo `definitions.py` contiene las estructuras compartidas para representar mensajes de sensores (`SensorMessage`) y la enumeración de roles del sistema, que identifica los tipos de participantes: `admin`, `panel`, `visualizer`, `server` y `actuador`.

5.6 Componente de servidor y persistencia

El componente `src/server` implementa el servicio encargado de recibir, procesar y almacenar la información generada por los paneles.

La capa de persistencia utiliza modelos SQLAlchemy para representar las entidades principales del sistema:

- **Panel**: representa cada panel fotovoltaico simulado.
- **TipoMedicion**: define el tipo y unidad de cada variable registrada.
- **Medicion**: almacena los valores históricos junto con su marca temporal. Incluye la referencia a un panel y un tipo de medición.

La capa de servicios implementa las operaciones de registro y almacenamiento mediante las funciones:

- `registrar_panel`.
- `registrar_sensor`.
- `registrar_medicion`.

Para evitar registros duplicados se utiliza un patrón *get-or-create*, mediante el cual se consulta la existencia previa de una entidad antes de crear una nueva instancia.

El aprovisionamiento de la base de datos PostgreSQL se automatiza mediante *cloud-init* en la máquina virtual que aloja el broker, creando los usuarios, permisos y estructuras necesarias para la ejecución del sistema.

Finalmente, el componente `ServerClient` consume los mensajes MQTT publicados por los paneles, los procesa y almacena las mediciones en la base de datos, completando el flujo de datos desde la simulación hasta la persistencia. Adicionalmente, en caso de detectar un caso anómalo en los datos, sube un mensaje al tópico que escucha los actuadores con el id del panel que posee dicho dato.

6 Seguridad y confiabilidad incorporada

6.1 Autenticación cliente–broker

El broker MQTT Mosquitto se configura con la opción `allow_anonymous false`, deshabilitando las conexiones sin autenticación. La gestión de credenciales se realiza mediante un archivo de contraseñas (`password_file`), generado automáticamente durante el proceso de aprovisionamiento de la máquina virtual mediante el script `mosquitto_setup.py`.

Cada componente del sistema utiliza credenciales independientes asociadas a su rol dentro de la arquitectura:

- **admin**: administración del broker y configuración del sistema.
- **server**: recepción y procesamiento de datos para persistencia.
- **visualizer**: acceso de lectura para la visualización de métricas.
- **panel**: publicación de datos provenientes de los paneles simulados.
- **actuador**: recepción de comandos de control.

De esta manera, ningún cliente puede establecer una conexión con el broker sin proporcionar credenciales válidas.

6.2 Control de acceso mediante ACL MQTT

Además del mecanismo de autenticación, se implementó un sistema de listas de control de acceso (ACL) para limitar las operaciones permitidas sobre cada tópico MQTT.

Las reglas de autorización se definen según el rol del cliente, restringiendo tanto las operaciones de lectura (`subscribe`) como de escritura (`publish`). Por ejemplo, los clientes con rol **panel** únicamente pueden publicar información asociada a sus propios tópicos de telemetría, evitando que un nodo pueda modificar datos pertenecientes a otros paneles.

De forma similar, el rol **visualizer** posee permisos únicamente de lectura sobre los tópicos de datos, mientras que el rol **actuador** utiliza canales independientes para recibir alertas y enviar comandos de control.

Esta separación reduce el riesgo de accesos no autorizados y evita interferencias entre los distintos componentes del sistema.

6.3 Endurecimiento de la infraestructura de visualización

La máquina virtual destinada a la visualización mediante Grafana incorpora medidas adicionales de protección a nivel del sistema operativo.

Se configura un cortafuegos con política predeterminada de denegación (*deny-by-default*) utilizando UFW, permitiendo únicamente los servicios necesarios para la operación:

- acceso remoto mediante SSH;
- servicio web de Grafana mediante el puerto 3000/tcp.

Además, se implementa `fail2ban` para proteger el servicio SSH frente a intentos repetidos de autenticación. La configuración establece un máximo de tres intentos fallidos dentro de un intervalo de diez minutos, generando un bloqueo temporal de una hora para la dirección IP origen.

Finalmente, se deshabilita el acceso directo mediante el usuario `root`, reduciendo la superficie de ataque y obligando a utilizar cuentas con permisos controlados para la administración del sistema.

7 Pruebas y validación

Para la validación del sistema se desplegó la arquitectura utilizando cuatro máquinas virtuales independientes, cada una con un rol específico:

- Máquina de simulación de paneles fotovoltaicos.
- Máquina del broker MQTT.
- Máquina del servidor central.
- Máquina del actuador.

Con el objetivo de aproximar un entorno de comunicación más cercano a una red real, sobre la máquina que alojaba el broker MQTT se aplicaron condiciones artificiales de red mediante la herramienta *traffic control* de Linux. En particular, se configuró una latencia de 50 ms y jitter de 10m ms, una pérdida de paquetes del 5 % y una limitación del ancho de banda a 54 Mbit/s. Estas condiciones permitieron evaluar el comportamiento del sistema frente a retardos y pérdidas durante la transmisión de datos.

La validación del correcto funcionamiento de la arquitectura se realizó verificando que la información publicada por los nodos simuladores fuera recibida correctamente por el servidor. Para ello, se monitorearon los mensajes de confirmación y registros (*logs*) generados por los paneles durante la publicación de datos. Posteriormente, se comprobó que dichos datos fueran almacenados correctamente en la base de datos PostgreSQL del servidor, verificando así la comunicación extremo a extremo entre los distintos componentes del sistema.

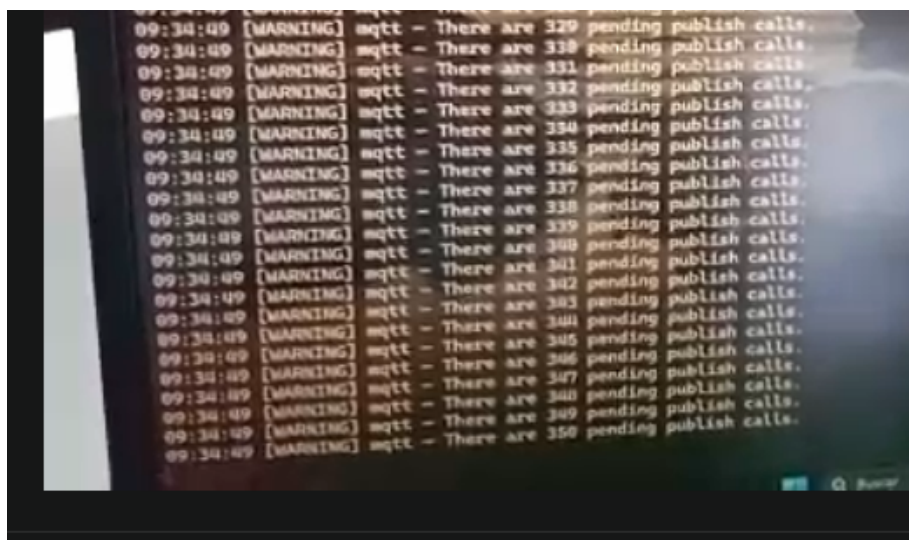


Figura 2: Ejecución de publicaciones de los 350 paneles solares.

id	panel_id	tipo_medicion_id	valor	timestamp
1	1	3	0	2026-07-07 07:35:11.930573
2	1	3	0.7181	2026-07-07 07:40:19.953479
3	1	2	0	2026-07-07 07:40:20.1205
4	1	1	12.0805	2026-07-07 07:40:20.128637
5	2	3	0.5208	2026-07-07 07:40:20.269882
6	2	2	0.4587	2026-07-07 07:40:20.401515
7	2	1	12.0002	2026-07-07 07:40:20.406674
8	3	3	1.134	2026-07-07 07:40:20.548949
9	3	2	0	2026-07-07 07:40:20.696017
10	3	1	12.446	2026-07-07 07:40:20.710045
11	4	3	1.2444	2026-07-07 07:40:20.864568
12	4	2	2.3442	2026-07-07 07:40:21.007271
13	4	1	11.8806	2026-07-07 07:40:21.021584
14	5	3	1.1383	2026-07-07 07:40:21.167349
15	5	2	1.9613	2026-07-07 07:40:21.298333
16	5	1	12.1497	2026-07-07 07:40:21.304778
17	6	3	0.4726	2026-07-07 07:40:21.709803
18	6	2	0	2026-07-07 07:40:21.843049
19	6	1	11.9375	2026-07-07 07:40:21.848236
20	7	3	1.4095	2026-07-07 07:40:23.034848
21	7	2	4.0928	2026-07-07 07:40:23.167878

Figura 1: Datos de paneles guardados en la base de datos

Asimismo, se emplearon diversas herramientas de diagnóstico de red para validar la conectividad y el funcionamiento de los servicios:

- **Ping:** se utilizó para verificar la conectividad IP entre las distintas máquinas virtuales y medir el tiempo de respuesta de la red.
- **Nmap:** permitió comprobar la disponibilidad de los puertos de los servicios desplegados, particularmente el puerto utilizado por el broker MQTT, verificando que estos fueran accesibles desde los distintos nodos de la arquitectura.
- **Wireshark:** se empleó para capturar y analizar el tráfico de red, verificando la transmisión de mensajes MQTT entre los clientes y el broker. Mediante esta herramienta fue posible observar los paquetes correspondientes a las operaciones de publicación (PUBLISH), sus respectivos acuses de recibo y el establecimiento de las conexiones TCP utilizadas por el protocolo.

En conjunto, estas pruebas permitieron validar tanto la correcta comunicación entre los componentes distribuidos como el funcionamiento de los mecanismos de transporte, autenticación y almacenamiento implementados en el sistema.

8 Análisis con Wireshark

8.1 Análisis de tráfico mediante Wireshark

Con el objetivo de validar el funcionamiento de la comunicación entre el broker, que recibe la información de los paneles simulados, y el servidor, se realizó una captura del tráfico de red utilizando Wireshark durante la operación normal del sistema. Con

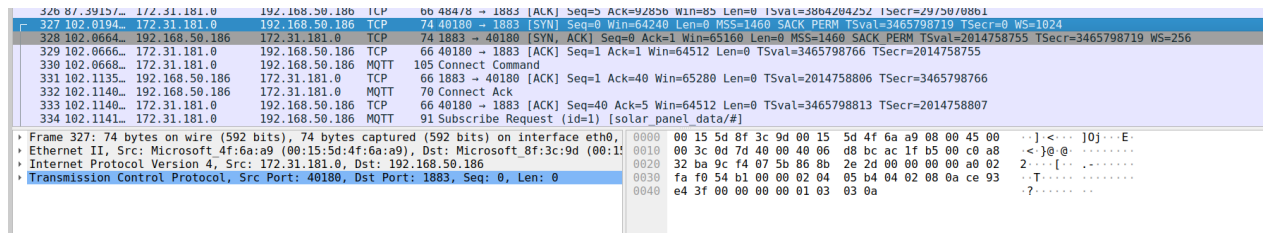


Figura 3: Captura de intercambio TCP para establecer la conexión entre un suscriptor (172.31.181.0) y el broker (192.168.50.186).

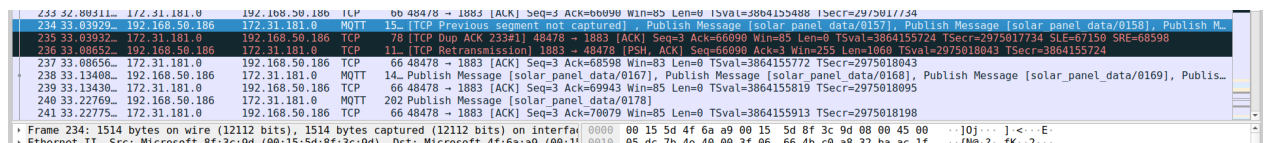


Figura 4: Captura de paquete perdido y solicitud para la retransmisión.

En la captura fue posible observar el establecimiento de conexiones TCP entre los clientes MQTT y el broker, verificándose el intercambio inicial de paquetes correspondiente al *three-way handshake* (SYN, SYN-ACK y ACK). Posteriormente, se identificaron los mensajes MQTT utilizados por la aplicación. Los paneles simulados publican periódicamente sus mediciones mediante mensajes MQTT PUBLISH. Sin embargo, dado que la captura se realizó sobre la interfaz correspondiente a la comunicación entre el broker y el servidor, el tráfico intercambiado entre los paneles y el broker no es visible en esta captura. En su lugar, se observa la comunicación TCP utilizada para transportar los mensajes MQTT entre ambos componentes.

En la Figura 4 se aprecia un ejemplo del mecanismo de confiabilidad de TCP frente a la pérdida de un segmento. El receptor detecta la ausencia de información en la secuencia de bytes recibida y responde mediante mensajes *Duplicate ACK*, indicando el último segmento recibido correctamente. Al detectar estos acuses de recibo duplicados, el emisor retransmite el segmento perdido (*TCP Retransmission*), permitiendo recuperar la información sin intervención de la aplicación. Este comportamiento evidencia los mecanismos de control de errores y retransmisión implementados por TCP para garantizar la entrega confiable de los datos.

Asimismo, se verificó que todos los paquetes MQTT fueron transportados sobre conexiones TCP persistentes, evitando el establecimiento de una nueva conexión para cada publicación y

reduciendo la sobrecarga del protocolo.

La captura también permitió comprobar que, a pesar de las condiciones de red configuradas (50 ms de latencia y 5% de pérdida de paquetes), la comunicación entre el broker y el servidor se mantuvo operativa gracias a los mecanismos de confiabilidad proporcionados por TCP, los cuales permitieron recuperar automáticamente los segmentos perdidos mediante retransmisiones.

En conjunto, el análisis realizado con Wireshark permitió visualizar el envío de mensajes a través de la red y corroborar el correcto funcionamiento del protocolo MQTT sobre TCP/IP, verificar la secuencia de publicación y confirmación de mensajes, y confirmar que la arquitectura implementada transporta correctamente la información generada por los paneles hacia el servidor.

9 Limitaciones del sistema

Sin cifrado en el transporte. Actualmente la comunicación MQTT utiliza el puerto 1883 sin cifrado TLS, por lo que el contenido de los mensajes puede ser inspeccionado mediante herramientas como Wireshark. Una mejora futura consiste en habilitar un listener sobre el puerto 8883 utilizando certificados X.509 para garantizar confidencialidad e integridad.

Simulación centralizada en un solo proceso. Los 350 paneles se calculan vectorialmente dentro de un único proceso, lo que favorece el rendimiento pero se aleja de una topología con un proceso o dispositivo físico por panel.

Endurecimiento asimétrico entre VMs. La VM del broker no cuenta, a la fecha, con la misma política de cortafuegos (UFW) ni con `fail2ban`, a diferencia de la VM de Grafana.

Simulación costosa. Debido a la arquitectura planteada y a la decisión de utilizar maquinas virtuales separadas para cada nodo del sistema, intentar simular el entorno completo en un solo dispositivo puede llegar a ser costoso y dependiente de usar más de un dispositivo en alguna LAN particular.

Base de datos expuesta. Para permitir la comunicación con Grafana de forma remota es necesario permitir que la base de datos escuche de otras direcciones IP, lo cual la vuelve mas vulnerable a ataques. Para frenar este riesgo se implementa el uso de permitir unicamente la IP que posea el servidor de Grafana además de una cuenta exclusiva de la base de datos que solo permita lectura. Lamentablemente, debido a que las IPs de los dispositivos de la LAN no son estáticas estas podrían cambiar y habría que actualizar las IPs a las que responde PostgreSQL. En un contexto empresarial se implementarían medidas como utilizar un VPN que permita una comunicación por túnel o una API que sirva para consultar la base de datos sin tener que pasar directamente por ella.

10 Conclusiones

El desarrollo de *Simulador SolarisMonitor* permitió construir, sobre una base íntegramente de software, un sistema de monitoreo remoto que cumple con el tema obligatorio del curso: un entorno ciberfísico simulado (la planta fotovoltaica), comunicación real sobre TCP/IP mediante MQTT, y al menos un mecanismo de seguridad efectivamente implementado (autenticación en el broker), junto con un mecanismo de confiabilidad genuino (el lazo de control cerrado por el actuador). El uso de cálculo vectorizado con NumPy permitió simular 350 paneles con un modelo físico razonablemente realista, con elementos como el ciclo día/noche por zona horaria, temperatura ambiente, ensuciamiento progresivo y eventos estocásticos de sombreado, todo esto sin comprometer el desempeño del sistema.

Como resultado del proyecto se logró comprender el funcionamiento del protocolo MQTT en mayor profundidad, investigando y entiendo las diversas herramientas que existen para su uso como lo fue *Mosquitto* en este caso, de manera que se pudo configurar de forma correcta los parámetros de red para un entorno simulado que se acerque a la realidad. Si bien el uso de este protocolo implicó reconocer y desarrollar un sistema algo diferente a como lo es el típico sistema basado en cliente-servidor, fue de gran utilidad para adentrarse de forma correcta en el mundo del internet de las cosas donde el poder de cómputo no es necesariamente su fuerte, entendiendo el funcionamiento y la simplicidad de sus paquetes y como es que se implementan las medidas de seguridad en este, que si bien, no se implementaron todas, si se alcanzó a desarrollar medidas que impidan que cualquiera pudiera adentrarse al sistema a enviar paquetes, siendo éste, una de los principales riesgos de seguridad previstos durante la concepción del proyecto.

A modo de cierre, durante el desarrollo de este trabajo no solo permitió para adentrarse de forma directa sobre el funcionamiento de la comunicación en red y aplicaciones sobre está, sino que también amplió el pensamiento crítico al momento de desarrollar la arquitectura de sistemas expuestos a la red, ya que si bien, el proyecto no está expuesto al internet en su totalidad, al momento de tomar decisiones siempre se tomaron en cuenta las consecuencias que podría tener centralizar o descentralizar componentes del sistema si fuera un entorno real, como lo fue, por ejemplo, el dilema inicial entre mantener el servicio de Grafana en un nodo distinto o el mismo del servidor central, durante la construcción del sistema se logró entender las consecuencias que hubiera tenido decidir entre una opción o la otra, en este caso la decisión final fue separarlos con el riesgo de tener que exponer la base de datos de cierta manera, pero que aun así, gracias a las medidas de seguridad implementadas, se llegó a que era posiblemente la mejor decisión arquitectónica para el proyecto. Este tipo de situaciones recalcó aún más el gran valor que tienen herramientas de ciberseguridad como lo son Wireshark, Nmap, y las VPN que podrían ser usadas en un contexto empresarial para evitar estar completamente expuesto al internet.

A Anexos

```
ar_panel_data/#"  
^Cvq@DESKTOP-3L37B00:~$ nmap 192.168.50.217  
Starting Nmap 7.95 ( https://nmap.org ) at 2026-07-07 09:13 -04  
Nmap scan report for lab2redesUbuntu.lan (192.168.50.217)  
Host is up (0.0092s latency).  
Not shown: 998 closed tcp ports (conn-refused)  
PORT      STATE SERVICE  
22/tcp    open  ssh  
5432/tcp   open  postgresql  
  
Nmap done: 1 IP address (1 host up) scanned in 1.34 seconds  
vq@DESKTOP-3L37B00:~$ █
```

Figura 5: Nmap al Servidor

```
vq@DESKTOP-3L37B00:~$ ping 192.168.50.217  
PING 192.168.50.217 (192.168.50.217) 56(84) bytes of data.  
64 bytes from 192.168.50.217: icmp_seq=1 ttl=63 time=4.67 ms  
64 bytes from 192.168.50.217: icmp_seq=2 ttl=63 time=4.11 ms  
64 bytes from 192.168.50.217: icmp_seq=3 ttl=63 time=4.59 ms  
64 bytes from 192.168.50.217: icmp_seq=4 ttl=63 time=4.31 ms  
64 bytes from 192.168.50.217: icmp_seq=5 ttl=63 time=4.30 ms  
64 bytes from 192.168.50.217: icmp_seq=6 ttl=63 time=4.05 ms  
64 bytes from 192.168.50.217: icmp_seq=7 ttl=63 time=3.89 ms  
64 bytes from 192.168.50.217: icmp_seq=8 ttl=63 time=3.60 ms  
^C  
--- 192.168.50.217 ping statistics ---  
8 packets transmitted, 8 received, 0% packet loss, time 7643ms  
rtt min/avg/max/mdev = 3.599/4.189/4.665/0.330 ms  
vq@DESKTOP-3L37B00:~$ █
```

Figura 6: Ping al Servidor

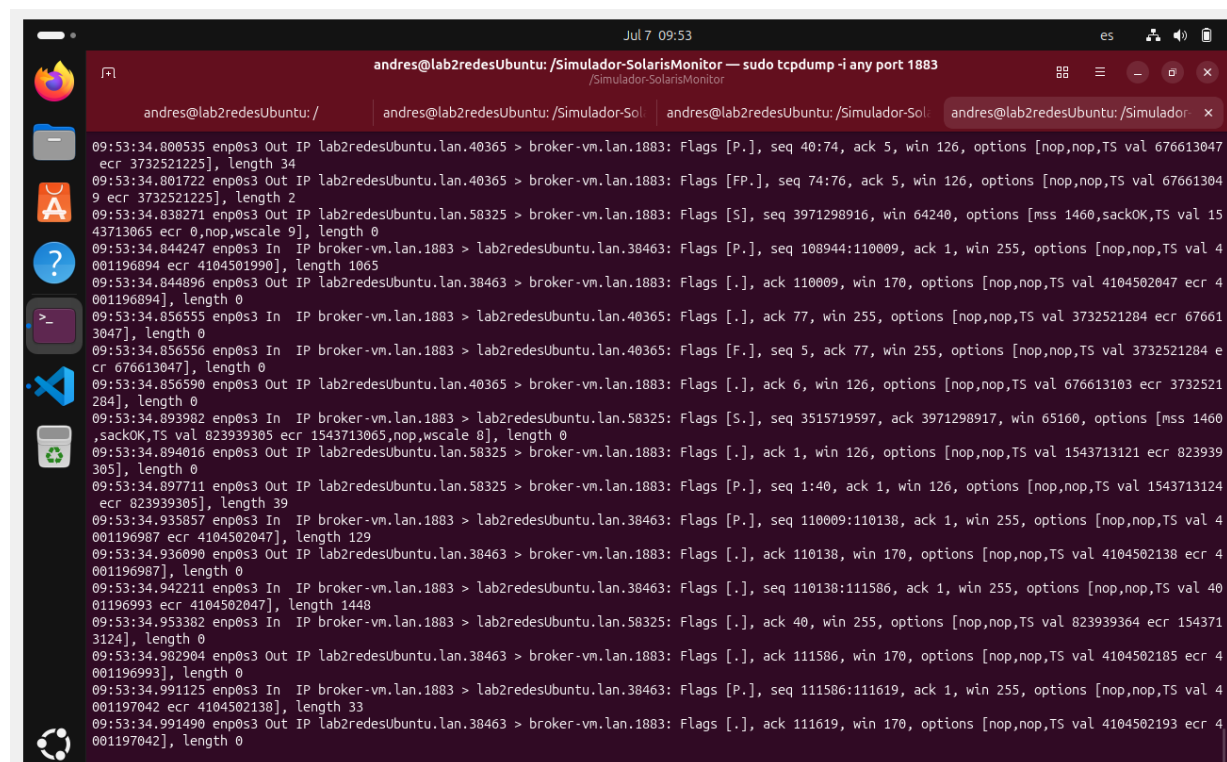
```

vq@DESKTOP-3L37B00:~$ nmap broker-vm
Starting Nmap 7.95 ( https://nmap.org ) at 2026-07-07 09:16 -04
Nmap scan report for broker-vm (192.168.50.186)
Host is up (0.052s latency).
rDNS record for 192.168.50.186: broker-vm.lan
Not shown: 999 closed tcp ports (conn-refused)
PORT      STATE SERVICE
22/tcp    open  ssh

Nmap done: 1 IP address (1 host up) scanned in 11.54 seconds
vq@DESKTOP-3L37B00:~$

```

Figura 7: Nmap al Broker



```

andres@lab2redesUbuntu: /Simulador-SolarisMonitor
andres@lab2redesUbuntu: /
andres@lab2redesUbuntu: /Simulador-Sol
andres@lab2redesUbuntu: /Simulador-Sol
andres@lab2redesUbuntu: /Simulador-

09:53:34.800535 enp0s3 Out IP lab2redesUbuntu.lan.40365 > broker-vm.lan.1883: Flags [P.], seq 40:74, ack 5, win 126, options [nop,nop,TS val 676613047 ecr 3732521225], length 34
09:53:34.801722 enp0s3 Out IP lab2redesUbuntu.lan.40365 > broker-vm.lan.1883: Flags [FP.], seq 74:76, ack 5, win 126, options [nop,nop,TS val 676613049 ecr 3732521225], length 2
09:53:34.838271 enp0s3 Out IP lab2redesUbuntu.lan.58325 > broker-vm.lan.1883: Flags [S.], seq 3971298916, win 64240, options [mss 1460,sackOK,TS val 1543713065,seq 3971298916,win 64240,ecn 0,nop,wscale 9], length 0
09:53:34.844247 enp0s3 In IP broker-vm.lan.1883 > lab2redesUbuntu.lan.38463: Flags [P.], seq 108944:110009, ack 1, win 255, options [nop,nop,TS val 4001196894,ecr 4104501990], length 1065
09:53:34.844896 enp0s3 Out IP lab2redesUbuntu.lan.38463 > broker-vm.lan.1883: Flags [.], ack 110009, win 170, options [nop,nop,TS val 4104502047 ecr 4001196894], length 0
09:53:34.856555 enp0s3 In IP broker-vm.lan.1883 > lab2redesUbuntu.lan.40365: Flags [.], ack 77, win 255, options [nop,nop,TS val 3732521284 ecr 676613047], length 0
09:53:34.856556 enp0s3 In IP broker-vm.lan.1883 > lab2redesUbuntu.lan.40365: Flags [F.], seq 5, ack 77, win 255, options [nop,nop,TS val 3732521284 ecr 676613047], length 0
09:53:34.856590 enp0s3 Out IP lab2redesUbuntu.lan.40365 > broker-vm.lan.1883: Flags [.], ack 6, win 126, options [nop,nop,TS val 676613103 ecr 3732521284], length 0
09:53:34.893982 enp0s3 In IP broker-vm.lan.1883 > lab2redesUbuntu.lan.58325: Flags [S.], seq 3515719597, ack 3971298917, win 65160, options [mss 1460,sackOK,TS val 823939305,ecr 1543713065,nop,wscale 8], length 0
09:53:34.894016 enp0s3 Out IP lab2redesUbuntu.lan.58325 > broker-vm.lan.1883: Flags [.], ack 1, win 126, options [nop,nop,TS val 1543713121 ecr 823939305], length 0
09:53:34.897711 enp0s3 Out IP lab2redesUbuntu.lan.58325 > broker-vm.lan.1883: Flags [P.], seq 1:40, ack 1, win 126, options [nop,nop,TS val 1543713124 ecr 823939305], length 39
09:53:34.935857 enp0s3 In IP broker-vm.lan.1883 > lab2redesUbuntu.lan.38463: Flags [P.], seq 110009:110138, ack 1, win 255, options [nop,nop,TS val 4001196987,ecr 4104502047], length 129
09:53:34.936090 enp0s3 Out IP lab2redesUbuntu.lan.38463 > broker-vm.lan.1883: Flags [.], ack 110138, win 170, options [nop,nop,TS val 4104502138 ecr 4001196987], length 0
09:53:34.942211 enp0s3 In IP broker-vm.lan.1883 > lab2redesUbuntu.lan.38463: Flags [.], seq 110138:111586, ack 1, win 255, options [nop,nop,TS val 4001196993,ecr 4104502047], length 1448
09:53:34.953382 enp0s3 In IP broker-vm.lan.1883 > lab2redesUbuntu.lan.58325: Flags [.], ack 40, win 255, options [nop,nop,TS val 823939364 ecr 1543713124], length 0
09:53:34.982904 enp0s3 Out IP lab2redesUbuntu.lan.38463 > broker-vm.lan.1883: Flags [.], ack 111586, win 170, options [nop,nop,TS val 4104502185 ecr 4001196993], length 0
09:53:34.991125 enp0s3 In IP broker-vm.lan.1883 > lab2redesUbuntu.lan.38463: Flags [P.], seq 111586:111619, ack 1, win 255, options [nop,nop,TS val 4001197042,ecr 4104502138], length 33
09:53:34.991490 enp0s3 Out IP lab2redesUbuntu.lan.38463 > broker-vm.lan.1883: Flags [.], ack 111619, win 170, options [nop,nop,TS val 4104502193 ecr 4001197042], length 0

```

Figura 8: Tráfico de red en la máquina del servidor